Record Linkage & Entity Resolution

Vu Tuyet Trinh
SoICT - HUST

1

Motivation example

- Merging two files …
  - Credit scoring
  - Customer lists
- Survey
  - CPS –Population Survey
  - SIPP – Survey Income
  - ❖Paper-based, interview,…
- Updating/unduplicating junk mail lists
  - Based on multiple sources of lists

2

Flight records: one entity or two?

TABLE 3.1: Sample Flights records

	AL	FN	DA	DD	DT	AA	AD	AT
r_{211}	A2	53	SFO	2014-02-08	15:35	EWR	2014-02-08	23:55
r_{212}	A2	53	SFO	2014-02-08	15:25	EWR	2014-02-08	00:05
r_{213}	A2	53	SFO	2014-02-08	15:27	EWR	2014-02-09	00:09
r_{214}	A1	53	SFO	2014-02-08	15:15	EWR	2014-02-08	23:30
r_{215}	A2	53	SFO	2014-03-08	15:27	EWR	2014-02-08	23:55
r_{221}	A2	53	SFO	2014-03-09	15:30	EWR	2014-03-09	23:45
r_{222}	A2	53	SFO	2014-03-09	15:37	EWR	2014-03-09	23:40
r_{223}	A2	53	SFO	2014-03-09	15:28	EWR	2014-03-09	23:37
r_{224}	A2	53	SFO	2014-03-08	15:25	EWR	2014-03-09	23:35
r_{231}	A1	49	EWR	2014-02-08	18:45	SFO	2014-02-08	21:40
r_{232}	A1	49	EWR	2014-02-08	18:30	SFO	2014-02-08	21:37
r_{233}	A1	49	EWR	2014-02-08	18:30	SAN	2014-02-08	21:30

Yellow records refer to one flight; red records to another.
Which values are errors, and which distinguish flights?

3

Record linking – Definition

- Find pairs of records in two files that represent the same entity
- Identify duplicate records within a file

4

Terminology

[Douglas Burdick, Ronald Fagin, Phokion G. Kolaitis, Lucian Popa, Wang-Chiew Tan: Expressive power of entity-linking frameworks. **J. Comput. Syst. Sci. 2019**]



- **Entity Linking**
  - "***Entity linking*** is the problem of creating links among records representing real-world entities that are related in certain ways."
  - "As an important special case, it includes **entity resolution**, which is the problem of identifying or linking duplicate entities
- **Other Terminology**
  - Entity Linking → Entity Linkage, Record Linkage
  - Entity Resolution → Data Deduplication, Entity Matching

☺

- **Applications**
  - Named entity recognition and disambiguation
  - Archiving, knowledge bases and graphs
  - Recommenders / social networks
  - Financial institutions (persons and legal entities)
  - Travel agencies, transportation, health care

Barack Obama
Barack **Hussein** Obama **II**
The US president (2016)

Barack and Michelle
are married ….

5

Problem

- **Entity Resolution**
  - "Recognizing those records in two files which represent identical persons, objects, or events"
  - Given two data sets A and B
  - Decide for all pairs of records $a_i - b_j$ in A x B if match (link), no match (non-link), or not enough evidence (possible-link)

[Ivan Fellegi, Alan Sunter: A Theory for Record Linkage, J. American. Statistical Assoc., pp. 1183-1210, **1969**]



- **Naïve Deduplication**
  - UNION DISTINCT via hash group-by or sort group-by
  - **Problem:** only exact matches

Name	Position	Affiliation	Research
Matthias Boehm	RSM	IBM Research – Almaden	Apache SystemML
Matthias Böhm	Prof	TU Graz	Apache SystemDS

→ **Similarity Measures**
- Token-based: e.g., Jaccard J(A,B) = (A ∩ B) / (A ∪ B)
- Edit-based: e.g., Levenshtein lev(A,B) → min(replace, insert, delete)
- Phonetic similarity (e.g., soundex, metaphone), Python lib Jellyfish

6

Similarity Mesures

7

Examples

- Typing and OCR errors
  - Ex: David Smith vs. Davod Smith
- Different formatting conventions:
  - Ex: 10/8 vs. Oct 8
- Custom abbreviation, shortening, or omission:
  - Ex: Daniel Walker Herbert Smith vs. Dan W. Smith
- Different names, nick names:
  - Ex: William Smith vs. Bill Smith
- Shuffling parts of strings:
  - Ex: Dept. of Computer Science, UW-Madison vs. Computer Science Dept., UW-Madison

8

8

Problem Description

- Given two sets of strings X and Y
  - Find all pairs x 2 X and y 2 Y that refer to the same real-world entity
  - We refer to (x,y) as a match
  - Example

Set X	Set Y	Matches
x_1 = Dave Smith	y_1 = David D. Smith	(x_1, y_1)
x_2 = Joe Wilson	y_2 = Daniel W. Smith	(x_3, y_2)
x_3 = Dan Smith		
(a)	(b)	(c)

- Two major challenges: accuracy & scalability

9

9

Accuracy Challenges

- Solution:
  - Use a similarity measure s(x,y) 2 [0,1]
    - The higher s(x,y), the more likely that x and y match
  - Declare x and y matched if s(x,y) ≥ t
- Distance measure/cost measure have also been used
  - Same concept
  - But smaller values ➔ higher similarities

10

10

Scalability Challenges

- Applying s(x,y) to all pairs is impractical
  - Quadratic in size of data
- Solution: apply s(x,y) to only most promising pairs, using a method FindCands
  - For each string x 2 X
    use method FindCands to find a candidate set Z µ Y
    for each string y 2 Z
    if s(x,y) ¸ t then return (x,y) as a matched pair
  - We discuss ways to implement FindCands later

11

11

Methods

- Sequence-based:
  - edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
- Set-based:
  - overlap, Jaccard, TF/IDF
- Hybrid:
  - generalized Jaccard, soft TF/IDF, Monge-Elkan
- Phonetic
  - Soundex

12

Edit Distance

- Also known as Levenshtein distance
- d(x,y) computes minimal cost of transforming x into y, using a sequence of operators, each with cost 1
  - Delete a character
  - Insert a character
  - Substitute a character with another
- Example: x = David Smiths, y = Davidd Simth, d(x,y) = 4, using following sequence
  - Inserting a character d (after David)
  - Substituting m by i
  - Substituting i by m
  - Deleting the last character of x, which is s

13

13

Edit Distance

- Models common editing mistakes
  - Inserting an extra character, swapping two characters, etc.
  - So smaller edit distance ➔ higher similarity
- Can be converted into a similarity measure
  - s(x,y) = 1 - d(x,y) / [max(length(x), length(y))]
  - Example
    - s(David Smiths, Davidd Simth) = 1 – 4 / max(12, 12) = 0.67

14

14

Computing Edit Distance using Dynamic Programming

- Define x = $\mathbf{x}_1\mathbf{x}_2\cdots\mathbf{x}_n$, y = $\mathbf{y}_1\mathbf{y}_2\cdots\mathbf{y}_m$
  - d(i,j) = edit distance between $\mathbf{x}_1\mathbf{x}_2\cdots\mathbf{x}_i$ and $\mathbf{y}_1\mathbf{y}_2\cdots\mathbf{y}_j$, the i-th and j-th prefixes of x and y
- Recurrence equations

$$d(i,j) = \min \begin{cases} d(i-1,j-1) & \text{if } x_i = y_j \quad \text{// copy} \\ d(i-1,j-1) + 1 & \text{if } x_i <> y_j \quad \text{// substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$d(i,j) = \min \begin{cases} d(i-1,j-1) + c(x_i,y_j) & \text{// copy or substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$c(x_i,y_j) = \begin{cases} 0 & \text{if } x_i = y_j, \\ 1 & \text{otherwise} \end{cases}$$

15

15

Example

- x = dva, y = dave

		y0	y1	y2	y3	y4
			d	**a**	**v**	**e**
x0		0	1	2	3	4
x1	**d**	1	0	1		
x2	**v**	2				
x3	**a**	3				

		y0	y1	y2	y3	y4
			d	**a**	**v**	**e**
x0		0	1	2	3	4
x1	**d**	1	0	1	2	3
x2	**v**	2	1	1	1	2
x3	**a**	3	2	1	2	2

x = d – v a
| | | |
y = d a v e

substitute a with e
insert a (after d)

- Cost of dynamic programming is O(|x||y|)

16

16

Set-based Similarity Measures

- View strings as sets or multi-sets of tokens
- Use set-related properties to compute similarity scores
- Common methods to generate tokens
  - consider words delimited by space
    - possibly stem the words (depending on the application)
    - remove common stop words (e.g., the, and, of)
    - e.g., given "david smith" ➔ generate tokens "david" and "smith"
  - consider q-grams, substrings of length q
    - e.g., "david smith" ➔ the set of 3-grams are {##d, #da, dav, avi, …, h##}
    - special character # is added to handle the start and end of string

17

17

The Overlap Measure

- Let $\mathbf{B}_x$ = set of tokens generated for string x
- Let $\mathbf{B}_y$ = set of tokens generated for string y
- $O(x,y) = |\mathbf{B}_x \cap \mathbf{B}_y|$
  - returns the number of common tokens
- E.g., x = dave, y = dav
  - $\mathbf{B}_x$ = {#d, da, av, ve, e#}, $\mathbf{B}_y$ = {#d, da, av, v#}
  - O(x,y) = 3

18

18

The Jaccard Measure

- $J(x,y) = |\mathbf{B}_x \cap \mathbf{B}_y| / |\mathbf{B}_x \cup \mathbf{B}_y|$
- E.g., x = dave, y = dav
  - $\mathbf{B}_x$ = {#d, da, av, ve, e#}, $\mathbf{B}_y$ = {#d, da, av, v#}
  - J(x,y) = 3/6
- Very commonly used in practice

19

19

The TF/IDF Measure: Motivation

- uses the TF/IDF notion commonly used in IR
  - two strings are similar if they share distinguishing terms
  - e.g., x = Apple Corporation, CA
    y = IBM Corporation, CA
    z = Apple Corp
  - s(x,y) > s(x,z) using edit distance or Jaccard measure, so x is matched with y ➔ incorrect
  - TF/IDF measure can recognize that Apple is a distinguishing term, whereas Corporation and CA are far more common ➔ correctly match x with z

20

20

Term Frequencies and Inverse Document Frequencies

- Assume x and y are taken from a collection of strings
- Each string is coverted into a bag of terms called a document
- Define term frequency tf(t,d) = number of times term t appears in document d
- Define inverse document frequency idf(t) = N / $\mathbf{N}_d$, number of documents in collection devided by number of documents that contain t
  - note: in practice, idf(t) is often defined as log(N / $\mathbf{N}_d$), here we will use the above simple formula to define idf(t)

21

21

Example

x = aab ⇨ B_x = {a, a, b}
y = ac ⇨ B_y = {a, c}
z = a ⇨ B_z = {a}

tf(a, x) = 2
tf(b, x) = 1
...
tf(c, z) = 0

idf(a) = 3/3 = 1
idf(b) = 3/1 = 3
idf(c) = 3/1 = 3

22

22

Feature Vectors

- Each document d is converted into a feature vector $\mathbf{v}_d$
- $\mathbf{v}_d$ has a feature $\mathbf{v}_d(\mathbf{t})$ for each term t
  - value of $\mathbf{v}_d(\mathbf{t})$ is a function of TF and IDF scores
  - here we assume $\mathbf{v}_d(\mathbf{t})$ = tf(t,d) * idf(t)

x = aab ⇨ B_x = {a, a, b}
y = ac ⇨ B_y = {a, c}
z = a ⇨ B_z = {a}

tf(a, x) = 2 idf(a) = 3/3 = 1
tf(b, x) = 1 idf(b) = 3/1 = 3
... idf(c) = 3/1 = 3
tf(c, z) = 0

	a	b	c
v_x	2	3	0
v_y	1	0	3
v_z	1	0	0

23

23

TF/IDF Similarity Score

- Let p and q be two strings, and T be the set of all terms in the collection
- Feature vectors $\mathbf{v}_p$ and $\mathbf{v}_q$ are vectors in the |T|-dimensional space wher each dimension corresponds to a term
- TF/IDF score of p and q is the cosine of the angle between $\mathbf{v}_p$ and $\mathbf{v}_q$
  - $s(p,q) = \sum_{t \in T} v_p(t) * v_q(t) \; / [\sqrt{\sum_{t \in T} v_p(t)^2} * \sqrt{\sum_{t \in T} vq(t)^2}]$

24

24

TF/IDF Similarity Score

- Score is high if strings share many frequent terms
  - terms with high TF scores
- Unless these terms are common in other strings
  - i.e., they have low IDF scores
- Dampening TF and IDF as commonly done in practice
  - use v_d(t) = log(tf(t,d) + 1) * log(idf(t)) instead of
    v_d(t) = tf(t,d) * idf(t)
- Normalizing feature vectors
  - v_d(t) = v_d(t) / $\sqrt{\sum_{\{t \in T\}} v_d(t)^2}$

25

25

Generalized Jaccard Measure

- Jaccard measure
  - considers overlapping tokens in both x and y
  - a token from x and a token from y must be identical to be included in the set of overlapping tokens
  - this can be too restrictive in certain cases
- Example:
  - matching taxonomic nodes that describe companies
  - "Energy & Transportation" vs. "Transportation, Energy, & Gas"
  - in theory Jaccard is well suited here, in practice Jaccard may not work well if tokens are commonly mispelled
    - e.g., energy vs. eneryg
  - generalized Jaccard measure can help such cases

26

26

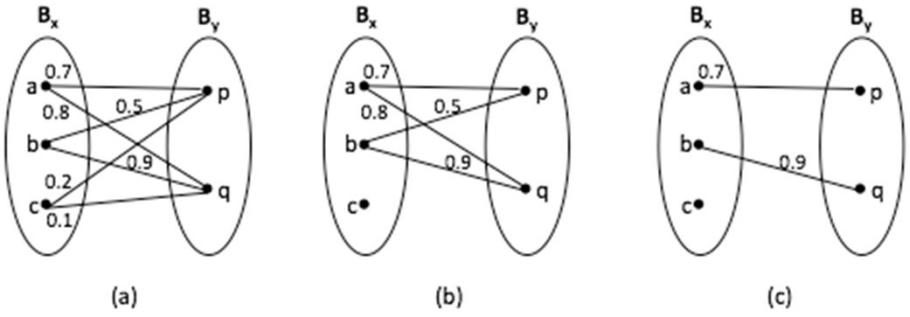
Generalized Jaccard Measure

- Let $\mathbf{B}_x = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, $\mathbf{B}_y = \{\mathbf{y}_1, \ldots, \mathbf{y}_m\}$
- Step 1: find token pairs that will be in the "softened" overlap set
  - apply a similarity measure s to compute sim score for each pair $(\mathbf{x}_i, \mathbf{y}_j)$
  - keep only those score **>=** a given threshold α, this forms a bipartite graph G
  - find the maximum-weight matching M in G
- Step 2: return normalized weight of M as generalized Jaccard score
  - $GJ(x,y) = \sum_{(xi,yj) \in M} \mathbf{s}(\mathbf{x}_i,\mathbf{y}_j) / (|\mathbf{B}_x| + |\mathbf{B}_y| - |M|)$

27

27

An Example



- Generalized Jaccard score: (0.7 + 0.9)/(3 + 2 – 2) = 0.53

28

28

The Soft TF/IDF Measure

- Using TF/IDF measure as the "higher-level" sim measure
  - e.g., "Apple Corporation, CA", "IBM Corporation, CA", and "Aple Corp", with Apple being mispelt in the last string
- Step 1: compute close(x,y,k): set of all terms $t \in \mathbf{B}_x$ that have at least one close term $u \in \mathbf{B}_y$, i.e., $s'(t,u) >= k$
  - s' is a basic sim measure (e.g., Jaro-Winkler), k prespecified
- Step 2: compute s(x,y) as in traditional TF/IDF score, but weighing each TF/IDF component using s'
  - $s(x,y) = \sum_{t \in close(x,y,k)} \mathbf{v}_x(\mathbf{t}) * \mathbf{v}_y(\mathbf{u}^*) * s'(t,u^*)$
  - $u^* \in \mathbf{B}_y$ maximizes $s'(t,u)$ 8 u2 $\mathbf{B}_y$

29

29

An Example

x = abcd

y = aa'b'cd'

(a)

B_x = {a, b, c, d}

1 0.7 0.8 1 0.6

B_y = {a, a', b', c, d'}

close(x, y, 0.75) = {a, b, c}

(b)

$s(x,y) = v_x(a) \cdot v_y(a) \cdot 1 +$
$v_x(b) \cdot v_y(b') \cdot 0.8 +$
$v_x(c) \cdot v_y(c) \cdot 1$

(c)

30

30

Phonetic Similarity Measures

- Match strings based on their sound, instead of appearances
- Very effective in matching names, which often appear in different ways that sound the same
  - e.g., Meyer, Meier, and Mire; Smith, Smithe, and Smythe
- Soundex is most commonly used

31

31

The Soundex Measure

- Used primarily to match surnames
  - maps a surname x into a 4-letter code
  - two surnames are judged similar if share the same code
- Algorithm to map x into a code:
  - Step 1: keep the first letter of x, subsequent steps are performed on the rest of x
  - Step 2: remove all occurences of W and H. Replace the remaining letters with digits as follows:
    - replace B, F, P, V with 1, C, G, J, K, Q, S, X, Z with 2, D, T with 3, L with 4, M, N with 5, R with 6
  - Step 3: replace sequence of identical digits by the digit itself
  - Step 4: Drop all non-digit letters, return the first four letters as the soundex code

32

32

The Soundex Measure

- Example: x = Ashcraft
  - after Step 2: A226a13, after Step 3: A26a13, Step 4 converts this into A2613, then returns A261
  - Soundex code is padded with 0 if there is not enough digits
- Example: Robert and Rupert map into R163
- Soundex fails to map Gough and Goff, and Jawornicki and Yavornitzky
  - designed primarily for Caucasian names, but found to work well for names of many different origins
  - does not work well for names of East Asian origins
    - which uses vowels to discriminate, Soundex ignores vowels

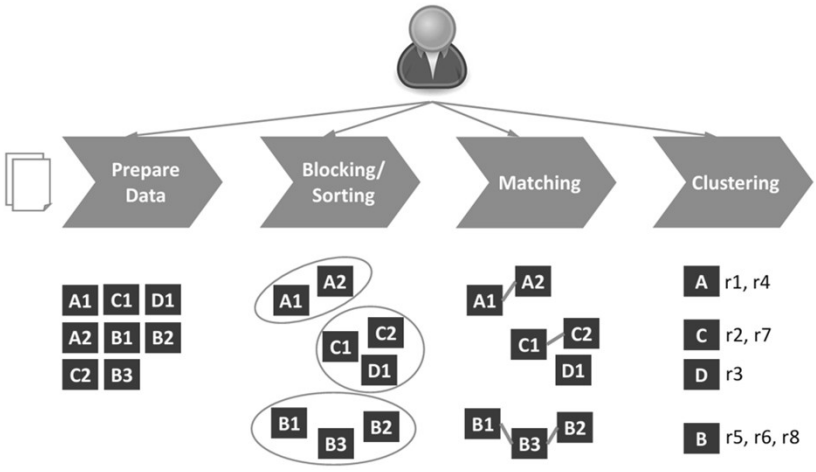
33

33

Entity Resolution

34

Entity Resolution Pipeline



35

Entity Linking Approaches

[Xin Luna Dong, Theodoros Rekatsinas: Data Integration and Machine Learning: A Natural Synergy. **PVLDB 2018**]

50 Years of Entity Linkage



Rule-based and stats-based
- Blocking: e.g., same name
- Matching: e.g., avg similarity of attribute values
- Clustering: e.g., transitive closure, etc.

Supervised learning
- Random forest for matching
  F-msr: >95% w. ~1M labels
- Active learning for blocking & matching
  F-msr: 80%-98% w. ~1000 labels

1969 (Pre-ML) — **~2000 (Early ML)** — **~2015 (ML)** — **2018 (Deep ML)**

Sup / Unsup learning
- Matching: Decision tree, SVM
  F-msr: 70%-90% w. 500 labels
- Clustering: Correlation clustering, Markov clustering

Deep learning
- Deep learning
- Entity embedding

36

Data Preparation

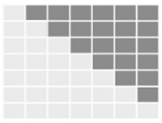
- Schema Matching and Mapping
  - Create homogeneous schema for comparison
  - Split composite attributes
- Normalization
  - Removal of special characters and white spaces
  - Stemming
  - Capitalization (to upper/lower)
  - Remove redundant works, resolve abbreviations
- Data Cleaning
  - Correct data corruption and inconsistencies

Autonomous, heterogeneous systems

37

Blocking & Sorting

- Naïve All-Pairs
  - Brute-force, naïve approach
  - ➢n*(n-1)/2 pairs O(n2) complexity



- Blocking / Partitioning
  - Efficiently create small blocks of similar records for pair-wise matching
  - Basic: equivalent values on selected attributes (name)
  - Predicates:whole field, token field, common integer, sam e x char start, n-grams
  - Hybrid: disjunctions/conjunctions
  - ..

38

Blocking & Sorting (2)

- Sorted Neighborhood
  - Define sorting keys (similar to blocking keys)
  - Sort records by sorting keys
  - Define sliding window of size m (e.g., 100) and compute all-pair matching within sliding window
- Blocking via Word Embeddings and LSH
  - Compute word/attribute embeddings + tuple embeddings
  - Locality-Sensitive Hashing (LSH) for blocking
  - K hash functions h(t)
  - k-dimensional hash-code
  - L hash tables, each k hash functions

Distributed Tuple Representation

39

Matching

- **Basic Similarity Measures**
  - Pick similarity measure sim(r, r') and thresholds: high θ_h (and low θ_l)
  - Record similarity: avg attribute similarity
  - **Match:** $sim(r, r') > \theta_h$ **Non-match:** $sim(r, r') < \theta_l$ **possible match:** $\theta_l < sim(r, r') < \theta_h$

- **Learned Matchers (Traditional ML)**
  - **Phase 1:** Learned string similarity measures for selected attributes
  - **Phase 2:** Training matching decisions from similarity metrics
  - Selection of samples for labeling (sufficient, suitable, **balanced**)
  - **SVM** and **decision trees, logistic regression, random forest**, XGBoost

[Mikhail Bilenko, Raymond J. Mooney: Adaptive duplicate detection using learnable string similarity measures. **KDD 2003**]



[Hanna Köpcke, Andreas Thor, Erhard Rahm: Evaluation of entity resolution approaches on real-world match problems. **PVLDB 2010**]



[Xin Luna Dong: Building a Broad Knowledge Graph for Products. **ICDE 2019**]



40

Matching (2)

- **Deep Learning for ER**
  - Automatic representation learning from text (avoid feature engineering)
  - Leverage pre-trained word embeddings for semantics (no syntactic limitations)

- **Example DeepER**

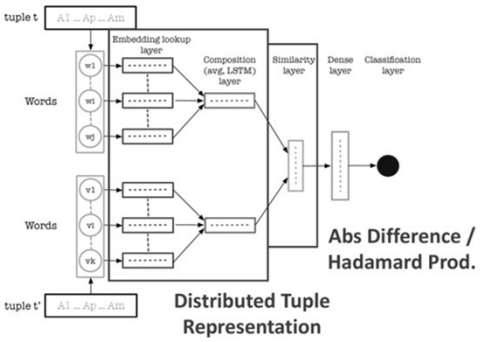


[Muhammad Ebraheem et al: Distributed Representations of Tuples for Entity Resolution. **PVLDB 2018**]

- **Example Magellan**
  - DL for text and dirty data



[Sidharth Mudgal et al: Deep Learning for Entity Matching: A Design Space Exploration. **SIGMOD 2018**]

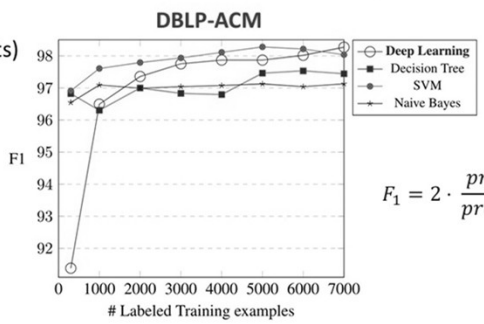


41

Matching (3)

[Sairam Gurajada, Lucian Popa, Kun Qian, Prithviraj Sen: Learning-Based Methods with Human in the Loop for Entity Resolution, Tutorial, **CIKM 2019**]

- **Labeled Data**
  - Scarce (experts)
  - Class skew





$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

➔ **Transfer Learning**
  - Learn model from high-resource ER scenario (w/ regularization)
  - Fine-tune using low-resource examples

➔ **Active Learning**
  - Select instances for tuning to min labeling

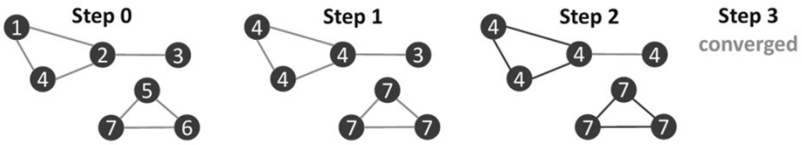
[Jungo Kasai et al: Low-resource Deep Entity Resolution with Transfer and Active Learning. **ACL 2019**]



42

Clustering

- **Recap: Connected Components**
  - Determine connected components of a graph (subgraphs of connected nodes)
  - Propagate max(current, msgs) if != current to neighbors, terminate if no msgs



- **Clustering Approaches**
  - Basic: connected components (transitive closure) w/ edges sim > θ_h
    → Issues: **big clusters** and **dissimilar records**
  - Correlation clustering: +/- cuts based on sims → global opt NP-hard
  - Markov clustering: stochastic flow simulation via random walks

[Oktie Hassanzadeh, Fei Chiang, Renée J. Miller, Hyun Chul Lee: Framework for Evaluating Clustering Algorithms in Duplicate Detection. **PVLDB 2009**]



43

Incremental Data Deduplication

- **Goals**
  - Incremental stream of updates
    → previously **computed results obsolete**
  - Same or similar results AND significantly faster than batch computation

[Anja Gruenheid, Xin Luna Dong, Divesh Srivastava: Incremental Record Linkage. **PVLDB 2014**]



- **Approach**
  - End-to-end incremental record linkage for new and changing records
  - Incremental maintenance of similarity graph and incremental graph clustering
  - Initial graph created by correlation clustering
  - Greedy update approach in polynomial time
    - Directly connect components from increment ΔG into Q
    - Merge of **pairs of clusters** to obtain better result?
    - Split of **cluster into two** to obtain better result?
    - Move nodes **between two clusters** to obtain better result?

44

References

- Dong & Srivastava, Big Data Integration, Chapter 3
- Peter Christen, Data Matching: Concepts and Techniques for Record Linkage
- Fellegi & Sunter, 1969, “A Theory for Record Linkage”
- Winkler, 1995, “Matching and Record Linkage,” in B.G. Cox et. al. (ed.), Business Survey Methods, New York, J. Wiley,

45



46